

LLD - Design Patterns

Software design patterns are important tools developers, providing proven solutions to common problems encountered during software development.

- Reusable solutions for typical software design challenges are known as design patterns.
- Provide a standard terminology and are specific to particular scenarios and problems.
- Not finished code but templates or blueprints only
- Software engineers use these to write more structured, manageable, and scalable code.

Key Characteristics of Design Patterns

- **Reusability:** Patterns can be applied to different projects and problems, saving time and effort in solving similar issues.
- **Standardization:** They provide a shared language and understanding among developers, helping in communication and collaboration.
- **Efficiency:** By using these popular patterns, developers can avoid finding the solution to same recurring problems, which leads to faster development.
- **Flexibility:** Patterns are abstract solutions/templates that can be adapted to fit various scenarios and requirements.

Why Learn Design Patterns?

- Code that is simpler to comprehend, update, and expand is produced with the help of design patterns.
- They offer solutions that have been tried and tested as well as best practices.
- Learning this enables them to quickly and effectively address similar challenges in various projects.
- Developers can produce reusable components that can be utilized in a variety of applications by implementing design patterns.
- This reduces redundancy and saves development time.

Types of Software Design Patterns

There are three types of Design Patterns:

- Creational Design Pattern
- Structural Design Pattern
- Behavioral Design Pattern

1. Creational Design Patterns

Creational Design Patterns focus on the process of object creation or problems related to object creation. They help in making a system independent of how its objects are created, composed and represented.

Types of Creational Design Patterns:

- **Factory Method Design Pattern**

- This pattern is typically helpful when it's necessary to separate the construction of an object from its implementation.
- With the use of this design pattern, objects can be produced without having to define the exact class of object to be created.

- **Singleton Method Design Pattern**

The Singleton Design pattern is the most straightforward to understand.

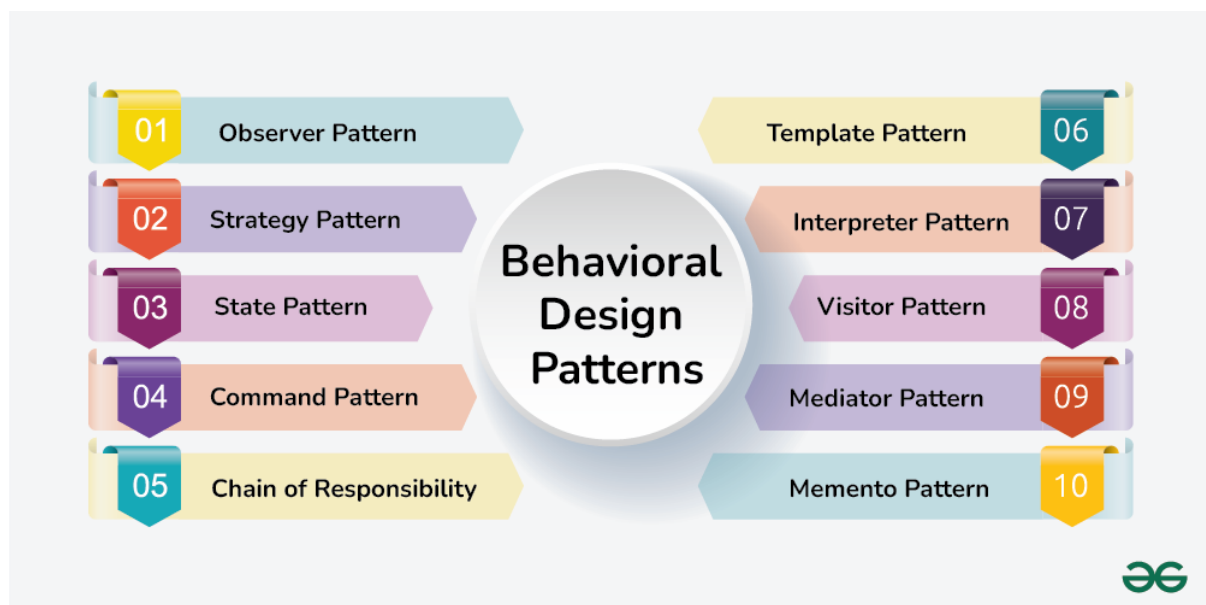
- It guarantees that a class has just one instance and offers a way to access it globally.

2. Structural Design Patterns

[Structural Design Patterns](#) solves problems related to how classes and objects are composed/assembled to form larger structures which are efficient and flexible in nature. Structural class patterns use inheritance to compose interfaces or implementations.

3. Behavioral Design Patterns

[Behavioral Patterns](#) are concerned with algorithms and the assignment of responsibilities between objects. Behavioral patterns describe not just patterns of objects or classes but also the patterns of communication between them. These patterns characterize complex control flow that's difficult to follow at run-time.



Types of Behavioral Design Patterns:

- **Observer Method Design Pattern**

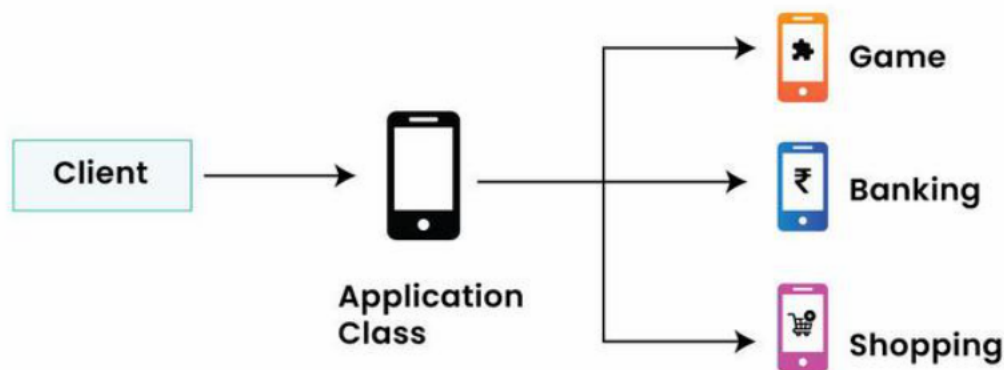
- It establishes a one-to-many dependency between objects, meaning that all of the dependents (observers) of the subject are immediately updated and notified when the subject changes.
- **Strategy Method Design Pattern**
 - It is possible to select an object's behavior at runtime by utilizing the Strategy Design Pattern.
 - Encapsulating a family of algorithms into distinct classes that each implement a common interface is the foundation of the Strategy pattern.

Factory Method Design Pattern (Virtual Constructor)

1. Core Concept

The Factory Method is a **creational design pattern** that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created. It promotes loose coupling by eliminating the need to bind application-specific classes into the code.

What is Factory Method Design Pattern



2. Problem it Solves

Imagine a logistics management application. Initially, it only handles transportation by trucks, so your code is full of direct instantiations of the Truck class (e.g., `new Truck()`). If you later need to add support for ships, you would have to modify all those parts of the code, violating the **Open/Closed Principle**.

3. Solution with Factory Method

The pattern suggests replacing direct object construction calls with calls to a special **factory method**. The objects returned by this method are often referred to as "products." You create a common interface for all products and a common creator class with the factory method.

- **Product:** Declares the interface common to all objects that the creator and its subclasses can produce.
- **Concrete Product:** Different implementations of the Product interface.

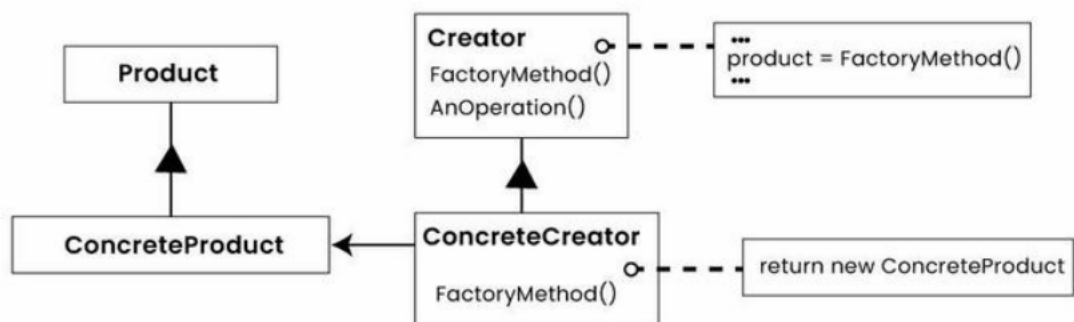
- **Creator:** Declares the factory method (createTransport()) that returns a Product object. It may also define a default implementation of the factory method.
- **Concrete Creator:** Overrides the factory method to return a different type of Concrete Product.

The client code works with the Creator and Product interfaces, making it independent of the specific concrete classes.

4. When to Use

- When a class cannot anticipate the class of objects it must create.
- When a class wants its subclasses to specify the objects it creates.
- When you want to provide a library of products and only reveal their interfaces, not their implementations.

Key Component of Factory Method Design Pattern in Java



5. Implementation Steps (Based on GeeksforGeeks Example)

The example models a logistics system (Creator) that can create different types of transport (Product: Truck, Ship).

Step 1: Create the Product Interface

// Transport.java (The Product Interface)

```
public interface Transport {
    void deliver();
}
```

Step 2: Create Concrete Products

// Truck.java (A Concrete Product)

```
public class Truck implements Transport {
    @Override
    public void deliver() {
```

```

        System.out.println("Delivering by land in a box.");
    }
}

// Ship.java (Another Concrete Product)
public class Ship implements Transport {
    @Override
    public void deliver() {
        System.out.println("Delivering by sea in a container.");
    }
}

```

Step 3: Create the Creator Abstract Class

- The createTransport() method is the **Factory Method**.

```

// Logistics.java (The Creator Abstract Class)
public abstract class Logistics {
    // Factory Method - to be implemented by subclasses
    public abstract Transport createTransport();

    // A common business logic method that uses the product
    public void planDelivery() {
        // Call the factory method to create a Transport object
        Transport t = createTransport();
        // Use the product (interface, not concrete class)
        t.deliver();
    }
}

```

Step 4: Create Concrete Creators

- Each Concrete Creator decides which Concrete Product to instantiate.

```

// RoadLogistics.java (A Concrete Creator)
public class RoadLogistics extends Logistics {
    @Override

```

```

    public Transport createTransport() {
        return new Truck(); // Creates the Concrete Product: Truck
    }
}

// SeaLogistics.java (Another Concrete Creator)

public class SeaLogistics extends Logistics {
    @Override
    public Transport createTransport() {
        return new Ship(); // Creates the Concrete Product: Ship
    }
}

```

Step 5: Client Code / Demo Class

- The client code works with the Creator (Logistics) and Product (Transport) interfaces. It is decoupled from the specific Truck and Ship classes.

```

// FactoryMethodDesignPattern.java

public class FactoryMethodDesignPattern {
    public static void main(String[] args) {
        // The type of logistics is chosen at runtime (e.g., via config, user input)
        Logistics roadLogistics = new RoadLogistics();
        roadLogistics.planDelivery(); // Output: Delivering by land in a box.

        Logistics seaLogistics = new SeaLogistics();
        seaLogistics.planDelivery(); // Output: Delivering by sea in a container.
    }
}

```

6. Key Advantages

- **Loose Coupling:** The client code is decoupled from concrete product classes. It only needs to know the abstract Logistics and Transport interfaces.
- **Single Responsibility Principle:** Product creation code is moved to one place (the Concrete Creator), making the code easier to support.
- **Open/Closed Principle:** You can introduce new types of products (e.g., Airplane) and new creators (e.g., AirLogistics) **without breaking existing client code**.

7. Important Considerations / Disadvantages

- The code may become more complicated as you need to introduce many new subclasses (Concrete Creators) to implement the pattern. It's often a trade-off between structure and simplicity.

The Factory Method Pattern defines a creation interface (createTransport()) and lets subclasses decide which specific class to instantiate, promoting flexibility and maintainability.

Advantages of Factory Method Design Pattern

Below are the main advantages of factory method:

- Separates object creation from client code, enhancing flexibility and maintainability since changes to creation don't affect clients.
- New product types can be easily added without altering client code by simply creating new Concrete Creator subclasses.
- Simplifies unit testing by allowing mock product creation, enabling tests of various implementations without actual object dependencies.
- The factory method can be reused across different application parts, centralizing and streamlining object creation logic.
- Hides specific product classes from clients, reducing dependencies and improving maintainability.

Disadvantages of Factory Method Design Pattern

Below are the main advantages of factory method:

- Adds more classes and interfaces, which can complicate understanding and maintenance, especially for newcomers.
- Polymorphism and dynamic binding may slightly affect performance, though this is usually minimal.
- Concrete creators remain closely linked to their products, necessitating changes across both when one is modified.
- Client code must be aware of concrete subclasses to make accurate factory calls.
- The pattern should be applied carefully to avoid unnecessary complexity; simple object creation may not need a factory.
- Testing the factory logic can be more complicated compared to simpler designs.

Singleton Design Pattern in Java

1. Core Concept

The Singleton pattern is a **creational design pattern** that ensures a class has only **one instance** and provides a **global point of access** to that instance. It is used for logging, driver objects, caching, thread pools, and database connections.

2. Problem it Solves

It solves two main problems:

1. **Controlled Access to a Single Instance:** It ensures that other classes can't create new instances of the singleton class, guaranteeing that all other classes use the same instance.
2. **Global Access Point:** It provides a well-known access point to the instance, avoiding the need to pass the object around as a parameter throughout the application.

3. Key Implementation Points

To implement a singleton class, the following points are crucial:

- **Private Constructor:** Restricts the instantiation of the class from outside the class.
- **Private Static Instance:** A static variable of the same class that is the only instance of the class.
- **Public Static Factory Method:** A static method that returns the instance of the class, providing the global access point.

4. Basic Implementation (Eager Initialization)

This is the simplest method, where the instance is created at the time of class loading.

Advantages:

- Simple and thread-safe by default.

Disadvantages:

- The instance is created even if the client application might not be using it, potentially wasting resources

```
// Singleton class
```

```
public class Singleton {
```

```
    // Private static instance of the class (eagerly initialized)
```

```
    private static final Singleton instance = new Singleton();
```

```
    // Private constructor to prevent instantiation
```



```

private Singleton() {}

// Public static method to provide global access
public static Singleton getInstance() {
    return instance;
}

// Other methods (example)
public void showMessage() {
    System.out.println("Hello from Singleton!");
}
}

// Client code
public class Main {
    public static void main(String[] args) {
        // Get the only object available
        Singleton object = Singleton.getInstance();

        // Show the message
        object.showMessage(); // Output: Hello from Singleton!

        // All calls to getInstance() return the same object
        Singleton anotherObject = Singleton.getInstance();
        System.out.println(object == anotherObject); // Output: true
    }
}

```

5. Lazy Initialization (Not Thread-Safe)

The instance is created only when it is requested for the first time.

Advantages:

Avoids unnecessary instance creation until it is needed.

Disadvantages:

Not thread-safe. If multiple threads enter the `getInstance()` method simultaneously when the instance is null, multiple instances can be created, breaking the singleton pattern.

```
public class LazySingleton {  
  
    private static LazySingleton instance;  
  
    private LazySingleton() {}  
  
    public static LazySingleton getInstance() {  
        if (instance == null) { // Check 1  
            instance = new LazySingleton(); // Not thread-safe here  
        }  
        return instance;  
    }  
}
```

6. Thread-Safe Implementations

a) Synchronized Method

Making the `getInstance()` method synchronized ensures that only one thread can execute it at a time.

Advantages:

- Simple and provides thread safety.

Disadvantages:

- **Poor performance** because every call to `getInstance()` is synchronized, even after the instance is already created, causing unnecessary locking overhead.

```
public class ThreadSafeSingleton {  
  
    private static ThreadSafeSingleton instance;  
  
    private ThreadSafeSingleton() {}  
  
    // Synchronized method to control simultaneous access  
    public static synchronized ThreadSafeSingleton getInstance() {  
        if (instance == null) {  
            instance = new ThreadSafeSingleton();  
        }  
        return instance;  
    }  
}
```

b) Double-Checked Locking

This approach synchronizes only the block of code responsible for creating the instance and uses an additional null check inside the synchronized block.

Advantages:

- **High performance.** Synchronization overhead is only for the first few threads when the instance is null.
- Thread-safe.

Disadvantages:

- Slightly more complex implementation.

```
public class DoubleCheckedLockingSingleton {

    // The 'volatile' keyword ensures changes are visible to all threads
    private static volatile DoubleCheckedLockingSingleton instance;

    private DoubleCheckedLockingSingleton() {}

    public static DoubleCheckedLockingSingleton getInstance() {
        if (instance == null) { // First check (non-synchronized for performance)
            synchronized (DoubleCheckedLockingSingleton.class) {
                if (instance == null) { // Second check (synchronized)
                    instance = new DoubleCheckedLockingSingleton();
                }
            }
        }
        return instance;
    }
}
```

8. Best Practice & Real-World Use

- For most modern applications, **Eager Initialization** is often sufficient and preferred for its simplicity, unless the class is resource-heavy.
- If lazy loading is absolutely necessary, **Double-Checked Locking** is the standard high-performance, thread-safe solution.
- The Singleton pattern is widely used in core Java libraries (e.g., `java.lang.Runtime#getRuntime()`) and frameworks like Spring (where beans are singletons by default).

Observer Design Pattern in Java

1. Core Concept

The Observer pattern is a **behavioral design pattern** that defines a **one-to-many dependency** between objects. When one object (the **Subject**) changes its state, **all its dependents (Observers)** are notified and updated automatically. It is also known as the **Publish-Subscribe** pattern.

2. Problem it Solves

It addresses scenarios where an object needs to notify an open-ended number of other objects about changes in its state. Without this pattern, the subject would need to be tightly coupled to all its dependent objects, making the system inflexible, hard to maintain, and violate the **Open/Closed Principle**.

3. Solution

The pattern suggests two main components:

1. **Subject:** Maintains a list of its dependents (observers) and provides methods to **add**, **remove**, and **notify** them.
2. **Observer:** Provides an update interface for objects that should be notified of changes in a subject.

This creates a loose coupling between the subject and its observers. The subject knows nothing about the concrete classes of the observers, only that they implement the Observer interface.

4. Key Participants / UML Structure

- **Subject (Interface/Abstract Class):**
 - +attach(Observer o)
 - +detach(Observer o)
 - +notifyObservers()
- **ConcreteSubject:**
 - -state (The data that observers are interested in)
 - +getState() (Returns the state)
 - +setState(state) (When state changes, calls notifyObservers())
- **Observer (Interface):**
 - +update()
- **ConcreteObserver:**
 - -subject (A reference to the subject it observes)
 - +update() (Gets the state from the subject and acts on it)

5. Implementation Example: News Agency System

A news agency (subject) notifies multiple channels (observers) whenever there is breaking news.

Step 1: Create the Observer Interface

Defines the contract for all observers.

```
// Observer.java
public interface Observer {
    public void update();
}
```

Step 2: Create the Subject Interface / Abstract Class

Defines the contract for the subject.

```
// Subject.java
import java.util.ArrayList;
import java.util.List;

public interface Subject {
    // Methods to register and unregister observers
    public void registerObserver(Observer obj);
    public void unregisterObserver(Observer obj);

    // Method to notify all observers of change
```

```
    public void notifyObservers();  
}
```

Step 3: Create the ConcreteSubject Class

Implements the Subject interface and holds the state.

```
// NewsAgency.java  
public class NewsAgency implements Subject {  
    private List<Observer> observers;  
    private String news;  
  
    public NewsAgency() {  
        this.observers = new ArrayList<>();  
    }  
  
    @Override  
    public void registerObserver(Observer obj) {  
        observers.add(obj);  
    }  
  
    @Override  
    public void unregisterObserver(Observer obj) {  
        observers.remove(obj);  
    }  
  
    @Override  
    public void notifyObservers() {  
        for (Observer obj : observers) {  
            obj.update(); // Notify each observer  
        }  
    }  
  
    // This method triggers the notification  
    public void setNews(String news) {  
        this.news = news;  
        notifyObservers(); // Critical: Notify observers on state change  
    }  
  
    public String getNews() {  
        return this.news;  
    }  
}
```

Step 4: Create ConcreteObserver Classes

Implement the Observer interface and react to updates from the subject.

```
// NewsChannel.java  
public class NewsChannel implements Observer {  
    private String news;  
    private Subject agency;
```

```

// The observer registers itself with the subject upon creation
public NewsChannel(Subject agency) {
    this.agency = agency;
    agency.registerObserver(this);
}

@Override
public void update() {
    // Pull the new data from the subject
    this.news = ((NewsAgency) agency).getNews();
    display(); // Take action
}

public void display() {
    System.out.println("News Channel Update: " + news);
}
}

```

Step 5: Client Code / Demo

```

// Client.java
public class Client {
    public static void main(String[] args) {
        NewsAgency agency = new NewsAgency(); // The Subject

        // Create observers and pass them the subject
        NewsChannel channel1 = new NewsChannel(agency);
        NewsChannel channel2 = new NewsChannel(agency);

        // Change the state of the subject -> triggers notifications
        agency.setNews("Breaking News: Observer Pattern Explained!");
        // Output:
        // News Channel Update: Breaking News: Observer Pattern Explained!
        // News Channel Update: Breaking News: Observer Pattern Explained!

        // An observer can unregister/detach itself
        agency.unregisterObserver(channel1);

        agency.setNews("Second News: Channel 1 is no longer listening.");
        // Output:
        // News Channel Update: Second News: Channel 1 is no longer listening.
    }
}

```

6. Push vs. Pull Models

- **Push Model:** The subject sends the detailed update (the data) as a parameter to the `update()` method. Observers are passive.
- **Pull Model (Used in the example):** The subject just sends a notification. It is the observer's responsibility to "pull" the specific data it needs from the subject using the subject's getter methods (like `getNews()`). This is more flexible.

7. Advantages

- **Loose Coupling:** The subject only knows that observers implement the Observer interface. It doesn't know their concrete classes.
- **Dynamic Relationships:** Observers can be added or removed at runtime.
- **Broadcast Communication:** A subject can notify an unlimited number of observers.
- **Open/Closed Principle:** You can introduce new observer classes without changing the subject's code.

8. Disadvantages

- **Unexpected Updates:** Since the subject has no knowledge of its observers' details, a change can trigger a cascade of updates which might be hard to debug or be inefficient.
- **Memory Leaks:** If observers are not properly unregistered, they cannot be garbage collected, leading to memory leaks.

9. Real-World Applications in Java

- **Java Swing / JavaFX:** `EventListener` models (e.g., `ActionListener`, `MouseListener`).
- **Java's Built-in Support:** The `java.util.Observer` interface and `java.util.Observable` class (now deprecated, but the pattern remains fundamental).
- **MVC Architecture:** The View (observer) updates when the Model (subject) changes.
- **Reactive Programming:** Libraries like `RxJava` are built upon the principles of the Observer pattern.

Strategy Design Pattern in Java

1. Core Concept

The Strategy pattern is a **behavioral design pattern** that enables selecting an algorithm's behavior at **runtime**. It defines a family of algorithms, encapsulates each one, and makes them **interchangeable**. This pattern lets the algorithm vary independently from the clients that use it.

2. Problem it Solves

It addresses situations where a class:

- Contains multiple conditional statements (e.g., if-else or switch-case) to choose between different variants of an algorithm.
- Needs to implement many related but slightly different behaviors.

This leads to a class that is:

- **Monolithic** and hard to maintain.
- **Inflexible;** adding a new algorithm requires modifying the existing class, violating the **Open/Closed Principle**.

- **Hard to test** due to multiple execution paths.

3. Solution

The Strategy pattern suggests:

1. **Extracting Algorithms:** Take the algorithms that vary and encapsulate each one into its own separate class. These are called **strategies**.
2. **Common Interface:** Define a common interface for all these strategy classes.
3. **Composition over Inheritance:** The original class, now called the **context**, does not implement the algorithm itself. Instead, it **holds a reference** to a strategy object and **delegates** the execution to it.

This allows the context to be independent of the concrete algorithm implementation.

4. Key Participants / UML Structure

- **Strategy (Interface):** Declares an interface common to all supported algorithms. The context uses this interface to call the algorithm defined by a concrete strategy.
- **ConcreteStrategy (Classes):** Implements the algorithm using the Strategy interface.
- **Context (Class):**
 - Is **configured** with a ConcreteStrategy object.
 - Maintains a **reference** to a Strategy object.
 - May define an interface that lets the Strategy access its data.
 - **Delegates** the execution of the algorithm to the linked strategy object.

5. Implementation Example: Payment System

A shopping cart needs to process payments using different methods (Credit Card, PayPal, UPI).

Step 1: Create the Strategy Interface

Defines the contract for all payment algorithms.

// PaymentStrategy.java

```
public interface PaymentStrategy {
    public void pay(int amount);
}
```

Step 2: Create ConcreteStrategy Classes

Implement the specific payment algorithms.

// CreditCardStrategy.java

```
public class CreditCardStrategy implements PaymentStrategy {
    private String cardNumber;
    private String cvv;

    public CreditCardStrategy(String cardNumber, String cvv) {
        this.cardNumber = cardNumber;
        this.cvv = cvv;
    }
```

@Override

```
public void pay(int amount) {
    System.out.println(amount + " paid with Credit Card.");
    // Logic to interact with credit card payment gateway
```



```

    }
}
// PayPalStrategy.java
public class PayPalStrategy implements PaymentStrategy {
    private String emailId;

    public PayPalStrategy(String emailId) {
        this.emailId = emailId;
    }

    @Override
    public void pay(int amount) {
        System.out.println(amount + " paid using PayPal.");
        // Logic to interact with PayPal API
    }
}

```

```

// UPIStrategy.java
public class UPIStrategy implements PaymentStrategy {
    private String upid;

    public UPIStrategy(String upid) {
        this.upid = upid;
    }

    @Override
    public void pay(int amount) {
        System.out.println(amount + " paid using UPI.");
        // Logic to interact with UPI system
    }
}

```

Step 3: Create the Context Class

Holds a reference to a strategy and delegates the work to it.

```

// ShoppingCart.java (The Context)
public class ShoppingCart {
    // Reference to the strategy
    private PaymentStrategy paymentStrategy;
    private int totalAmount = 0;

    public void addItem(int price) {
        totalAmount += price;
    }

    // The key method: Set the strategy at runtime
    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.paymentStrategy = strategy;
    }
}

```

```

// Delegate the payment task to the strategy object
public void checkout() {
    paymentStrategy.pay(totalAmount);
    totalAmount = 0; // Reset after payment
}
}

```

Step 4: Client Code / Demo

```

// Client.java
public class Client {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        cart.addItem(100);
        cart.addItem(200);

        // Select and set the payment strategy at RUNTIME
        cart.setPaymentStrategy(new CreditCardStrategy("1234-5678-9012", "123"));
        cart.checkout(); // Output: 300 paid with Credit Card.

        cart.addItem(50);
        // Easily switch to a different strategy
        cart.setPaymentStrategy(new PayPalStrategy("user@example.com"));
        cart.checkout(); // Output: 50 paid using PayPal.

        cart.addItem(500);
        cart.setPaymentStrategy(new UPIStrategy("user@upi"));
        cart.checkout(); // Output: 500 paid using UPI.
    }
}

```

6. How it Replaces Conditionals

Without Strategy (Problematic):

// BAD: Monolithic class with conditionals

```

public class ShoppingCart {
    ...
    public void checkout(String paymentMethod) {
        if ("CreditCard".equals(paymentMethod)) {
            // Process credit card payment
        } else if ("PayPal".equals(paymentMethod)) {
            // Process PayPal payment
        } // ... more else-if blocks for new methods
    }
}

```

With Strategy (Solution):

The if-else blocks are eliminated. The client simply chooses the appropriate strategy

object (new CreditCardStrategy(...)) and passes it to the context. The context's checkout() method becomes simple and stable.

7. Advantages

- **Open/Closed Principle:** New strategies can be added without changing the context or other existing strategies.
- **Eliminates Conditional Statements:** Removes large, repetitive conditional statements for algorithm selection.
- **Promotes Composition over Inheritance:** Favors object composition (having a strategy) over class inheritance (being a specific type), leading to more flexible code.
- **Improved Testability:** Each strategy is a separate class and can be tested in isolation. The context can be tested by injecting a mock strategy.

8. Disadvantages

- **Increased Number of Classes:** The pattern can lead to a proliferation of classes if there are many strategies.
- **Client Awareness:** The client must understand the differences between strategies to select the appropriate one.
- **Overhead:** Communication overhead between the context and the strategy may exist if the interface is not well-designed.

9. Real-World Applications in Java

- **java.util.Comparator:** The compare() method is a strategy for defining sorting orders. The Collections.sort() method is the context.
- **GUI Layout Managers:** In Swing, BorderLayout, FlowLayout, and GridLayout are strategies for arranging components within a container.
- **Spring Framework:** The PlatformTransactionManager interface has different strategies for transaction management (e.g., DataSourceTransactionManager, JtaTransactionManager).